

H0004 — Cargar competidores

Sistema de Gestión de Torneos Deportivos de Contacto (Taekwondo)

Épica: Inscripción y llaves

3 puntos

KANO: Básico

Rol: Coordinador

1. Qué se implementó

El **Coordinador** puede **cargar competidores** en un torneo y ver el listado de inscriptos. Cada competidor lleva sus datos personales, físicos, de escuela y responsable.

Decisión de modelo **revisada**: el competidor se carga **sin categoría**. La relación competidor↔categoría se resuelve al **armar las llaves (H0005)**, clasificándolo por sus atributos (sexo, edad, peso, graduación) contra los rangos de cada categoría — con posibilidad de revisar/ajustar categorías en ese momento.

Por qué el cambio: pedir la categoría en el alta obligaba a elegir a mano y **sin validar el encaje** (se podía meter un competidor de 14 años en una categoría de 10–12). Desacoplar refleja el flujo real (registrar atletas → clasificarlos) y evita ese error. Se agregó el campo `sexo` al competidor, imprescindible para clasificar contra el `sexo` de la categoría.

2. Backend (Clean Architecture + CQRS)

Cambios de modelo: `Competidor.CategoriaId` pasó a **nullable**, se agregó `Competidor.Sexo` (M/F), y la FK a categoría es opcional con `ON DELETE SET NULL`. Migración `CompetidorSinCategoriaYConSexo` (columna `sexo` con backfill "M", `categoria_id nullable`) — se auto-aplica al arrancar la API.

Archivo	Rol
Features/Competidores/CompetidorResponse.cs	DTO con <code>nombreCompleto</code> , <code>sexo</code> y <code>categoriaNombre</code> (hoy null hasta H0005).
ICompetidorData + CompetidorValidationRules	Reglas compartidas (incluye sexo) entre command y request.
Commands/CreateCompetidor/...	Command + Handler (torneo 404/409, carga sin categoría) + Validator.
Queries/GetCompetidoresByTorneo/...	Query + Handler del listado.
Infrastructure/.../CompetidorRepository.cs	EF Core (nuevo repositorio) + registro en DI.

Método	Ruta	Rol
GET	<code>/api/v1/torneos/{torneoId}/competidores</code>	Coordinador
POST	<code>/api/v1/torneos/{torneoId}/competidores</code>	Coordinador

Reglas de negocio en el alta. El torneo debe existir (404) y no estar `Finalizado` (409). La validación cubre datos

personales, sexo (M/F), edad (1–120), peso (>0 , ≤ 500 kg), altura (>0 , ≤ 3 m) y graduación (enum oficial de cinturones).

Flujo de una carga

```
POST /torneos/{id}/competidores (rol Coordinador)
-> CreateCompetidorRequestValidator (valida datos del competidor)
-> CreateCompetidorEndpoint (arma el Command y lo envia con MediatR)
-> ValidationBehavior (pipeline: valida el Command)
-> CreateCompetidorCommandHandler (torneo 404/409; crea SIN categoria)
-> CompetidorRepository.AddAsync (EF Core -> PostgreSQL)
-> 201 Created + CompetidorResponse
```

3. Frontend (React 19 + TanStack Query)

Archivo	Rol
<code>types/index.ts</code>	<code>Competidor</code> (categoría opcional, <code>sexo</code>) y <code>CreateCompetidorInput</code> .
<code>lib/api.ts</code> + <code>hooks/useCompetidores.ts</code>	<code>competidores.listar/cargar</code> ; hooks con query keys por torneo (invalida en <code>onSuccess</code>).
<code>components/competidores/CompetidorForm.tsx</code>	Alta (RHF + Zod) con select de sexo (ya no de categoría); resetea para cargar varios.
<code>components/competidores/CompetidorList.tsx</code>	Tabla semántica responsive (columna Sexo en lugar de Categoría).
<code>routes/torneos/\$torneoid/competidores.tsx</code>	Página (solo Coordinador) con aviso de que la categoría se asigna al armar las llaves.

4. Decisiones de diseño

Reglas de validación compartidas. La validación de competidor vive en `CompetidorValidationRules` sobre `ICompetidorData`, reutilizada por el validator del comando y el del request (sin duplicar).

Pendiente para H0005. Un atleta puede competir en Combate y Formas (categorías distintas); el único `categoria_id` por competidor no modela eso. Queda como pregunta de diseño de la clasificación en H0005.

5. Pruebas

- `CreateCompetidorCommandHandlerTests` — alta válida (sin categoría, con sexo), torneo inexistente (404), torneo finalizado (409).
- `CreateCompetidorCommandValidatorTests` — nombre vacío, sexo inválido, edad fuera de rango, graduación inválida, peso inválido, altura fuera de rango.

✓ Backend compila 0 errores · **54/54 tests** en verde.

✓ Frontend `tsc + vite build` sin errores.

Nota: verificación end-to-end pendiente (entorno Docker/backend detenido). La migración se auto-aplica al arrancar la API.

6. Checklist

Ítem	Estado
Competidor desacoplado de categoría (nullable) + campo sexo + migración	✓
Command + Handler + Validator (alta sin categoría)	✓
Query + Handler (listado) · Repositorio + DI	✓
Endpoints POST/GET con Roles("Coordinador")	✓
Reglas: torneo 404/409 · validación de datos (incl. sexo)	✓
Mapeo Mapster (.Adapt + with) · reglas compartidas	✓
Tests handler + validator	✓
Frontend: types + api + hook + form (sexo) + tabla + ruta	✓ responsive, WCAG 2.2 AA
Docs actualizadas (CLAUDE.md raíz + skill de BD)	✓
Verificación E2E	pendiente (entorno caído)